

CLASS: CMSC 443 - Cryptology

DATE: 14 May, 2026

TO: Dr. Chris Marron

FROM: Raiyaan Tareen, Aidan Denham

SUBJECT: LLM-Generated-Steganographic-PKC

1 ABSTRACT

Recent advancements in Large Language Models (LLMs) have transformed communication, yet their role in secure messaging remains underexplored, especially in surveillance-heavy environments [1]. Recent work in the field of steganography has begun focusing on the inclusion of AI/ML, especially LLM's, within novel cryptographic embedding frameworks. One such theoretical framework found in resource 1 proposes an LLM-agnostic approach for covertly embedding Public Key or Symmetric Key encrypted data within human-like texts to bypass restrictions on traditional encryption [1]. However, the practical viability of any such framework requires rigorous empirical validation. This project presents a functional proof-of-concept implementation of this novel cryptographic embedding framework. Developed in Python, our system operationalizes the theoretical model to empirically evaluate its real-world efficiency and legitimacy as a deployable product. We assess the computational overhead of the embedding process, the system's ability to operate using independent local LLMs, and the true indistinguishability of the generated ciphertext from natural human conversation. This implementation provides critical insights into the feasibility, scalability, and operational limits of LLM-driven steganography for secure communication in restrictive digital environments.

2 MOTIVATION

Based on the paper presented by Gligoroski, Raikwar, and Jha, it is believed that the possibility of using LLM's to generate steganographic streams in real time may be feasible. This paper examines such premises and attempts to utilize open source LLM's on a compute cluster provided by the High-Performance Computing Center at the University of Maryland, Baltimore County. If the results of our experiments show that real-time LLM embedded conversations are possible, this presents an attack vector on a global scale. Under such a paradigm, any AI-generated communication could potentially conceal a malicious payload beneath the transmitted message, while remaining effectively indistinguishable from benign AI-generated conversation to observers.

Take, for instance, the example of sharing nuclear launch codes. Two threat actors messaging over an onion network can use Gligoroski, Raikwar, and Jha's algorithm to simulate a conversation about any topic of their choosing, and embed a hidden payload about nuclear launch codes. Any predated algorithm analyzing dark web message channels has no

countermeasures for such a parameterized and dynamically-adjustable embedding, and this creates a national security incident.

It is for eventualities such as these which call for research to be done in these fields. Therefore, our contribution to this slice of history will be through creating a proof-of-concept conversation between human and LLM, then furthering the investigation into its effectiveness by using cryptographic tests to analyze the produced ciphertext. If baselines can be established which accurately measure the throughput of such an algorithm, it can be determined whether this type of attack is feasible. Further, measuring the effective randomness of the ciphertext character stream can further provide insight as to patterns, frequency distributions of characters, and otherwise indications that this type of attack has occurred.

3 METHODOLOGY

The methodology behind our project consists of explicit implementations of algorithms 1, 2, and 3 from the framework proposal paper followed by empirical testing using sampling methods, LLM selection, random payloads, and different context datasets.

3.1 Algorithm Implementation

The LLM embedding framework is broken down into four different algorithms. Algorithm 1 is called “EmbedderLLM”, and outlines the core functionality of the LLM interaction and story generation. The second algorithm is called “LLM Authentication Encryption” and is responsible for generating the parameters that are fed into EmbedderLLM ultimately producing our final desired story. Algorithm 3 is called “LLM Authenticated Decryption and Verification” and outlines the decryption process of a received generated story using a shared secret password. (I recommend skipping to 3.1.2 to understand the flow of code)

3.1.1 Algorithm 1

Algorithm 1 begins with the initialization of several variables which will be responsible for determining the fine tuning of token generation. These variables are dynamically adjusted by the algorithm logic to ensure meaningful token generation but are first chosen by the user as passed-in parameters which can simplify or enhance the accuracy and precision of generated stories.

Algorithm 1 EMBEDDERLLM(LLM, *TOPIC*, *Story*₀, *T*₀, *k*₀, **C**, **b**, *l*, *sec*)

Input: LLM model, *TOPIC* discussed in *Story*, initial content of story *Story*₀, initial values for temperature *T*₀ and *k*₀ for selection of top *k* tokens, sequence of characters **C** = [*C*₀, ..., *C*_{*n*-1}] and the sequence of integers **b** = [*b*₀, ..., *b*_{*n*-1}]. The value of *l* can be one from [1, 2, 3, 4] and determines which set *L_l* we use, and *sec* ∈ [32, 48, 64, 96, 128] where 2^{-*sec*} is the probability a used token to be produced with parameters outside the optimal ranges.

Output: A text string *Story*

```

1: i ← 0, n ← len(C), Story ← Story0, prev_pos ← len(Story)
2: Close ← False, T ← T0, k ← k0, Slow_Down ← 0
3: top_f ← Table[sec, l]
4: t_slow_down ←  $\frac{0.2}{21 \times \text{top\_f}}$ 

```

Figure 1: Algorithm 1 initialization steps pseudocode

Figure 1 a snapshot of the pseudocode written out in the paper and defines the passed in parameters as well as the first variable initializations. The parameters that are passed in are first generated by algorithm 2 (see 3.1.2) and are then stored as local variables. The first parameter is “LLM” which is simply the name of the desired LLM model that will be referenced in the API call. The second parameter is “TOPIC” which is a short sentence about what you want your generated stories to discuss. This could be something like “Plan a weekend trip” or “Optimal Syzygies of Celestial Bodies”. The decision of the topic is incredibly important in determining the efficiency of algorithm 1. If the former topic previously listed is chosen then the LLM will likely have plenty of possible tokens it can generate in order to continue the story onwards. If the latter much more niche topic is chosen, statistically speaking, the generated top token list will not only be incredibly constrained but will likely run into embedding issues with step 7 of the algorithm discussed later. The next two parameters chosen are “T0” & “k0” and are the initial values for LLM temperature and selection pool size for top k-tokens. The paper notes that for the text to remain indistinguishable from human chat, k should optimally stay between 40 and 60 & T should be between 0.7 and 0.9. The next two parameters are **C** and **b** which are sequences of characters & integers respectively. Both **C** & **b** are explained in depth in 3.1.2. The final two parameters are “l” and “sec”. The l parameter (which takes a value of 1, 2, 3, or 4) determines which specific set of characters *L_l*. The algorithm is allowed to be used when mapping the encrypted hexadecimal ciphertext into English characters. The sec parameter defines the maximum acceptable probability that the algorithm will generate a token that sounds “robotic” or distinguishable from human text. It typically takes values like 16, 32, 48, 64, 96, or 128.

```
# Embedder LLM function
# -- Parameters --
# LLM: name of specific LLM model
# TOPIC: Topic of story generated
# Story0: Possible previous story
# T0: Starting value for temperature
# k0: initial value for the top k0 tokens w/ top k0 Probs
# C: sequence = [C0, C1,..., C_(n-1)] of chars from some sets S1,...,S4
# b: sequence of int = [b0,b1,...,b_(n-1)] {same number n as C}
def embedderLLM (LLM, TOPIC, Story0, T0, k0, C, b, L, sec):

    # initialize all variables
    i = 0 # initial step
    n = len(C) # length of desired embedded text
    Story = Story0 # Prev story is curr story
    prev_pos = len(Story) # Index of last spot in story
    Close = False # emergency shut off for token placement
    T = T0 # Update Temp
    k = k0 # Update k
    Slow_Down = 0 # reset Slow_Down count for new iteration
    Unsuccessful = False

    # pick max num of repetitive attempts to find an appropriate token
    # before needing to increase the k param
    top_f = TOP_F_TABLE[sec, L]

    # Calculate Slow_Down Step
    tSlowDown = 0.2 / (21 * top_f)
```

Figure 2: Algorithm 1 initialization steps written in python

Figure 2 is a snippet of my (Raiyaan's) personal implementation of Figure 1. The first variable listed is "i" which just keeps track of the current step embedder is on in the while loop. The second variable "n" is the length of the desired embedded text; It's how long the **C** list is. After that we assign the passed in Story0 to Story and any changes or tokens added will be appended to this variable. Prev_pos is the index value of the last inserted value, this is to keep track of how far the next b_i index value is being placed from the edge of the story. Close is a boolean proximity alert and braking mechanism to prevent the algorithm from accidentally overshooting the next b_i index. The program is designed to add filler words occasionally to meet the "n" length desired quota but can sometimes add large filler words which add characters that overshoot that b_i index. When this occurs, "Close" is set to true and the program is essentially not allowed to add another filler word and keeps generating words that will eventually fit the b_i requirement via temperature increase. "T0" and "k0" are assigned to T & k to signify starting values for both. "Slow_down" is a counter variable which keeps track of how many times the algorithm has consecutively bumped up the temperature to try and escape the bottleneck. "Unsuccessful" is a boolean flag that indicates whether the current loop completely failed to make any forward progress after fine tuning. "Top_f" is the maximum number of retry attempts the algorithm is allowed to make incrementally, increasing *only* the temperature before it

absolutely must increase the token pool (k). “Top_f” is specifically chosen through a predesignated table defined within the paper; See Appendix A. “tSloDown” is a mathematically calculated increment step which the temperature is increased by during each retry attempt. The 0.2 numerator is the variance for the temperature range between 0.7 to 0.9 for human-like text. If the algorithm starts at 0.7 and fails repeatedly it can safely add ‘tSloDown’ exactly ‘top_f’ times and the final temperature value will only barely cross the 0.9 threshold. Next, we can start the actual while loop logic.

```
5: while  $i < n$  do
6:    $Y_{top-k} \leftarrow \text{LLM}(\text{TOPIC}, \text{Story}, T, k)$ 
7:    $Y = \{y \in Y_{top-k} \mid \text{Char}(\text{Story}||y, b_i) = C_i\}$ 
8:   if  $Y \neq \emptyset$  then
9:      $\text{next\_token} \xleftarrow{\$} Y$ 
10:     $\text{Story} \leftarrow \text{Story}||\text{next\_token}$ 
11:     $\text{prev\_pos} \leftarrow \text{len}(\text{Story})$ 
12:     $T \leftarrow T_0, k \leftarrow k_0$ 
13:     $\text{Slow\_Down} \leftarrow 0$ 
14:     $\text{Close} \leftarrow \text{False}$ 
15:     $i \leftarrow i + 1$ 
```

Figure 3: Algorithm 1 successful token generation embedding procedure

```
# -- MAIN LOOP --
while i < n:

    # Generate top list of tokens
    Y_topk = top_k_token_retriever(LLM, TOPIC, Story, T, k)

    # Check tokens for valid b_i positions
    Y_valid = token_pos_check(Y_topk, Story, C[i], b[i])

    # Check if Y_valid populated
    if len(Y_valid) > 0:

        # Append a RANDOM word from Y_valid list
        chosenOne = secrets.choice(Y_valid)
        Story = Story + chosenOne
        prev_pos = len(Story)

        # Reset values
        Close = False
        T = T0
        k = k0
        Slow_Down = 0

        # increment
        i += 1
```

Figure 4: Code implementation of Figure 3

Figure 3 and 4 are the pseudocode and actual implementation of the start of the while loop which actually iterates through the generation of the story. Line 5 starts the main loop: It will run until it successfully embeds all “n” characters of your cryptographic payload (C). The algorithm queries the local LLM. It passes the current Story, the TOPIC, and the current parameters for Temperature (T) and Top-k (k). The LLM returns a list of the k most probable next words or tokens that make sense in the context of the story (See Appendix B for LLM helper function code). Line 7 in figure 3 is condensed into a helper function called “token_pos_check” which is defined in Appendix C. in Figure 4 Y_valid receives the output of the helper function which returns a new, smaller set (Y) containing *only* the words that, when appended to the story, force the exact target character (C_i) to land perfectly on the exact target index (b_i). First we must check to see if Y_valid contains anything via the if statement found in figure 4. Any word in this new Y_valid list is a perfect fit as the next token to be appended to the story so we simply pick one at random using secrets.choice, append it to “Story” and update the “prev_pos” to reflect the new most recent C_i placement. It then resets the Close, T, k, and Slow Down parameters and increments our i counter. This is if everything goes smoothly, the following code is to account for any difficulties in generating tokens.

```

16:  else
17:      Y_shuffled ← RandomShuffle(Y_top-k)
18:      Unsuccessful ← True
19:      for next_token ∈ Y_shuffled do
20:          if len(Story||next_token) < bi - 6 then
21:              Story ← Story||next_token
22:              Unsuccessful ← False
23:              Break
24:          else if len(Story||next_token) < bi then
25:              if (Not Close) then
26:                  Close ← True
27:                  Story ← Story||next_token
28:                  Unsuccessful ← False
29:                  Break
30:              else
31:                  Slow_Down ← Slow_Down + 1
32:                  if Slow_Down < top_f then
33:                      Unsuccessful ← False
34:                      T ← T + t_slow_down
35:                      Break
36:              else
37:                  Slow_Down ← 0
38:              end if
39:          end if
40:      end for
41:      if Unsuccessful then
42:          Story ← Story[: prev_pos]
43:          T ← T + t_slow_down
44:          k ← k + 1
45:          Slow_Down ← 0, Close ← False
46:      end if
47:  end if
48: end if
49: end while

```

Figure 5: Pseudocode for Fallback/Filler Logic

```
76     else:
77
78         # shuffle the list of top-K & flip unsuccessful
79         Y_shuffle = Y_topk[:]
80         random.shuffle(Y_shuffle)
81         Unsuccessful = True
82
83         # iterate through each shuffled token to see if it fits special critrion
84         for next_token in Y_shuffle:
85             if (len(Story + next_token) < b[i] - 6):
86                 Story = Story + next_token
87                 Unsuccessful = False
88                 break
89
90             # append word if its not close & < b[i]
91             elif (len(Story + next_token) < b[i]):
92                 if not Close:
93                     Close = True
94                     Story = Story + next_token
95                     Unsuccessful = False
96                     break
97
98             # Last call, increment Slow_Down
99             Slow_Down += 1
100             if (Slow_Down < top_f):
101                 Unsuccessful = False
102                 T = T + tSloDown
103                 break
104
105             # reset Slow_Down
106             else:
107                 Slow_Down = 0
108
109         # After itterations & still unsuccessful, retry w/ +k
110         if Unsuccessful:
111             Story = Story[:prev_pos]
112             T = T + tSloDown
113             k += 1
114             Slow_Down = 0
115             Close = False
116
117     return Story
```

Figure 6: Implementation of Fallback/Filler Logic

Figures 5 & 6 are the pseudocode and exact python implementation for the Fallback/Filler Logic defined within the paper. All the code found in figure 6 is nested within a giant else statement that runs only when the code in figure 4 fails and none of the LLM generated tokens can naturally place our secret character at the required $b[i]$ index. Instead of giving up,

In figure 6, Lines 78-81 attempt to add a “filler” word to move the story forward, or it adjusts the parameters to force the LLM to try again. First, it makes a copy of the LLM's suggested words and shuffles them. This is to account for the text possibly becoming repetitive due to the likelihood of “depth” (top 1 token being the word “the” each time or something). Shuffling adds natural human-like variety. It also assumes this loop will fail by setting “Unsuccessful” to True.

For lines 84-88 in figure 6, the code essentially simulates adding the shuffled token to the story. If adding this token leaves the story length at least 6 characters away from your target index ($b[i]$), it is considered safe. It will then permanently add the token, flags the attempt as a success by changing unsuccessful back to False, and immediately breaks out of the loop so it can ask the LLM for a fresh set of words.

The next section of code is contained in lines 91-96 found in figure 6. If the token is too long for the safe zone, but appending it keeps the story *strictly before* the target index, it enters the danger zone. The Close flag is now checked and if it is currently false it simply adds the word and sets Close to True. On the next loop, because Close is now True, this if statement will fail, preventing the code from adding any more filler words and accidentally overshooting the target index.

Lines 98-107 deal with a simulated token that overshoots found in figure 6. If the simulated token overshoots the target index (meaning $len \geq b[i]$), or if the code is already in the danger zone (Close is True), it skips down to this block. Instead of adding a word it increments the Slow_Down counter and if you haven't hit your retry limit (top_f), it bumps up the Temperature ($T = T + tSloDown$), sets Unsuccessful = False since we aren't at retry limit, and breaks to go ask the LLM for more creative words.

Finally, in figure 6, lines 109-115 are reached if and only if the for loop finishes evaluating for EVERY shuffled token and *never* hits a break statement. This means no more filler words are left and the algorithm is completely stuck. At this point, it deletes the recent text back to the last saved “prev_pos” index, bumps up the temperature (T) again, expands the token pool by incrementing k by 1, and resetting Slow_Down & Close to try building the same sentence again from scratch but with wider parameters.

If all goes well and the while loop iterates until the very end without too many hiccups or getting stuck infinitely looping, Algorithm 1 finally returns the embedded story it just masterfully crafted.

3.1.2 Algorithm 2: LLM Authenticated Encryption

Algorithm 2, titled “LLM Authenticated Encryption”, serves as the overarching symmetric-key cryptographic protocol for this framework. Algorithm 2 handles the actual cryptography, preparing the data *before* it gets handed to the embedder. The full python implementation of Algorithm 2 is rather lengthy but performs exactly what will be described below using various python libraries such as “SHAKE-128” & “AESGCM”. Please refer to our Github for python implementation.

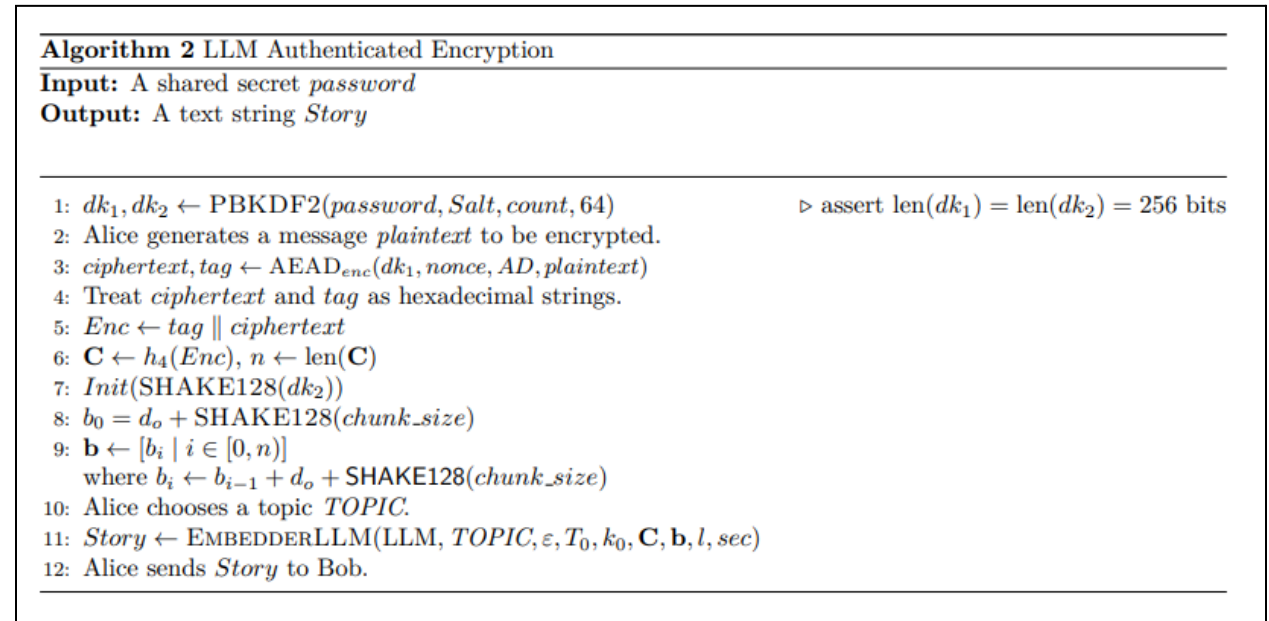


Figure 7: Pseudologic for full symmetric-key cryptographic encryption protocol

Figure 7 lays out the pseudologic for Algorithm 2. The first phase is the shared password generation. This is a critical step as without a properly generated shared password, a recipient will be unable to decrypt a received embedded story. In this case, let Alice and Bob share a secret password, Alice feeds this password into a key derivation function called PBKDF2. This function stretches the password to derive two distinct 256-bit keys: dk_1 & dk_2 . Splitting the keys here is crucial for security as one key is used to lock the message and the other to hide it.

Next, steps 2-6 in figure 7 define the encryption & character mapping of a potential message. Alice wants to send a secret plain text message, the algorithm then uses an Authenticated Encryption with Associated Data (AEAD) algorithm, unlocked by the first key (dk_1), to encrypt the message. This outputs the ciphertext and a secure checksum called a tag. The algorithm then joins the tag and ciphertext together into a single hexadecimal string called “Enc”. In step 6, the algorithm applies the h_4 mapping function defined in the paper. This maps

her hexadecimal string into the 16 most common characters in the English language (yielding the sequence **C**).

With Alice's string of common letters, **C**, the algorithm moves onto steps 7-9 where we create list **b** which is a deterministically random list of embedding placement integers. The algorithm initializes a pseudorandom number generator, in this case SHAKE128 using her second secret key, dk_2 . Because SHAKE128 is deterministic, anyone with dk_2 will generate the exact same sequence of random numbers. The algorithm uses these random numbers, combined with the offset spacing parameter (d_o), to generate the sequence **b**. This sequence or list is essentially the list of indices where the chars found in sequence **C** will be placed.

In Steps 4-12 found in figure 7, Alice can finally call 3.1.1 Algorithm 1, EmbedderLLM, to finally generate a story with the previously generated parameters. Alice simply picks a topic, and passes it into an "EmbedderLLM" call along with **C**, and **b** sequences. Once finished cooking, the story is spat out by algorithm 2 and ready to be copy pasted and sent to Bob who will now have to run the decryption algorithm 3.

3.1.3 Algorithm 3: LLM Authenticated Decryption & Verification

Algorithm 3, titled LLM Authenticated Decryption & Verification, is the receiver side of the symmetric key framework. The logic here actually perfectly mirrors 3.1.2 Algorithm 2 which Alice used to encrypt or hide the message. The best part is Bob does not need to run an LLM to extract the message. All Bob needs to do is generate the SHAKE128 random positions using the shared password and pluck the characters that are stored in the story string at those locations.

Algorithm 3 LLM Authenticated Decryption and Verification

Input: A shared secret *password* and a text string *Story*

Output: *plaintext* or *Fail*

```

1:  $dk_1, dk_2 \leftarrow \text{PBKDF2}(\text{password}, \text{Salt}, \text{count}, 64)$ 
2:  $Enc \leftarrow \text{empty string}$ 
3:  $\text{Init}(\text{SHAKE128}(dk_2))$ 
4:  $pos \leftarrow d_o + \text{SHAKE128}(\text{chunk\_size})$ 
5: while  $pos < \text{len}(\text{Story})$  do
6:    $Enc \leftarrow Enc + \text{Story}[pos]$ 
7:    $pos \leftarrow pos + d_o + \text{SHAKE128}(\text{chunk\_size})$ 
8: end while
9:  $tag \leftarrow Enc_0, \dots, Enc_{31}$ 
10:  $tag \leftarrow h_4^{-1}(tag)$ 
11:  $ciphertext \leftarrow Enc_{32}, \dots, Enc_{last}$ 
12:  $ciphertext \leftarrow h_4^{-1}(ciphertext)$ 
13:  $plaintext \leftarrow \text{AEAD}_{dec}(dk_1, \text{nonce}, AD, ciphertext, tag)$ 

```

Figure 8: Pseudologic for full symmetric-key cryptographic decryption protocol

In Figure 8, first, Bob uses the shared password and any other parameters between him and Alice to run PBKDF2 to regenerate the two 256-bit keys dk_1 & dk_2 .

From here, he creates an empty string called Enc to hold the secret characters he will find as he finds them. He initializes SHAKE128 using dk_2 which is the exact same dk_2 as Alice's, resulting in the same pseudorandom number generation. He then calculates pos, the index of the very first hidden character by adding the standard offset d_0 to the first random number chunk.

Now, Bob iterates through the ciphertext message to collect the payload. First, he starts a while loop that runs as long as his target position is less than the total length of the story. He iterates to the desired position in the story, grabs the character, and appends it to his encoded message.

Bob calculates the next target position using the same formula (adding the offset d_0 and the next random chunk from SHAKE128.) The while loop then repeats through the next chunk of text and chooses indices until the end of the story is reached.

Bob finally has Enc fully populated with common letters. From here he translates them back into cryptographic format. Bob splits his extracted characters into two groups: the first 32 characters become the tag, and the remaining characters become the ciphertext. He applies the inverse mapping function (h_4^{-1}) to both the tag and the ciphertext. This translates those common English letters back into their true hexadecimal forms.

Finally, he feeds the hexadecimal ciphertext and tag into the AEAD decryption function, using the first secret key (dk_1). If the data is pristine and the keys match, the function spits out the original secret plaintext. Else the tag will fail to check and the function will return fail.

4 RESULTS

Before the writing of this paper, it had been determined that this section would be filled with various conversations between ourselves and the EmbedderLLM, profiling performance as a function of latency across multiple open source models, and testing the integrity of data produced by each model. Examples of such tests span from cryptographic integrity testing; i.e. logging the success rate of lossless data extraction, randomness testing; i.e. measuring the log-odds of the produced ciphertext and comparing it to the frequency distribution of both random and english text, and linguistic legitimacy tests; using NLP tools to assess the “human-ness” of produced output.

However, after running the algorithm for days on end, the furthest into a story we were able to receive is 5 characters out of 52. See appendix D for the full conversation I, as a human being, appeal to my ethos and diagnose that this text, in all its glory, does **not** sound humanlike by any sense of the word. With that being said, there are positives: each character was accurately placed in the proper position, and *if a full story were to be produced*, decryption would be possible.

To validate the correctness of the steganographic encryption and decryption pipeline **independently** of any language model, a deterministic mock embedder is substituted for the live LLM during testing. The mock, RecordingPositionEmbedder, intercepts the character sequence $C = (C_0, C_1, \dots, C_{n-1})$ and the position sequences $b = (b_0, b_1, \dots, b_{n-1})$, that Algorithm 2 derives from the password and plaintext, constructs a string of length $b_{n-1}+1$ filled with a neutral padding character (english uppercase X), and writes C_i at index b_i for each i . This guarantees that the resulting story satisfies the embedding invariant $\text{story}[b_i] = C_i$ by construction, isolating all cryptographic and position-generation logic from the stochastic behaviour of the model.

The test is parameterised over five plaintext inputs of increasing size: a single byte, a short ASCII string, a sentence-length message, and a repeated paragraph totalling 180 bytes, and a 32-byte binary sequence, to exercise the algorithm across a range of embedding lengths n . For each case, `verify_story_correctness` is invoked with the generated story, the original password, and the original plaintext. The function re-derives the 256-bit key pair (dk_1, dk_2) via SHAKE-128, recomputes the position sequence from dk_2 , extracts the character at each position from the story, recovers the AES_GCM nonce from the first $2 \cdot \text{NONCE_SIZE}$ extracted hex characters, and re-encrypts the plaintext under dk_1 with that nonce to reconstruct the expected character sequence. A positional mismatch table is reported if any story $[b_i]$ disagrees with the expected character. Finally, `decrypt_from_story()` is called as a check: successful AEAD tag verification confirms that the full binary blob (nonce, ciphertext, and authentication tag) was embedded and extracted without corruption. All five variants pass both the per-character placement check and AEAD decryption, confirming end-to-end correctness of Algorithms 2 and 3 for plaintexts of arbitrary length. See appendix E for the full results of the test case

4 CONSIDERATIONS FOR IMPROVEMENT

The primary reason the results were so *funky* was due to the model it was being run on. The testing setup for this project was two-fold: A high-compute cluster at UMBC was being utilized, as well as a personal machine with a small graphics card attached to it. The compute cluster had the Gemma4:26b model loaded, which took too long to produce even a single character of output text. When run on the cluster, the following resources were reserved:

- I. 2 RTX_8000 Graphics Cards
- II. 30GB of VRAM (each RTX_8000 has 48)

And **still** after multiple days of running there was not a single character of output produced.

Eventually, the compute cluster was no longer a viable option, as too many concurrent users were utilizing the cluster and the next job could not be scheduled until well after the project was due. Lowering the computational power requirements did not help. Therefore, testing was migrated to an NVIDIA Jetson Orin Nano Super, with 7.6 total GB of shared memory. Clearly, the Gemma4 model could not fit on this small system, so instead the gemma3:1b model was used. This model only has 1 billion parameters, and is designed for edge computing rather than bulky tasks such as this. Therefore, it is expected that it will struggle with making the sentences coherent. However, it is assumed that using a larger model with thinking capabilities and more context would eliminate this issue. At such a point, natural language processing tools may be a viable humanness measurement option.

Additionally, the limited VRAM means that many runs became corrupted by OOM errors, and therefore the ciphertext produced was not accurate. On average, only one out of five algorithm runs produced even a single character, with exponentially less producing 2... and so on.

Therefore, the first and foremost consideration for improvement: own a data center. If this project was given funding, a dedicated graphics card would have been purchased to run bigger models on, without a scheduling wait time of 36+ hours. This, at the very least, would mean tests could run more quickly, produced ciphertext was of higher quality, and more iterations of attempted improvements could have occurred.

5 CONCLUSIONS

The results of this experiment were quite saddening from an accomplishment perspective, but insight was nonetheless gained into the feasibility of an LLM-generated Steganography attack. As this entire experiment has shown, current open source models and graphics cards make this type of attack **computationally infeasible** for the average cybercriminal. While specialized hardware may make this attack possible sometime in the future, it is significantly quicker to manually construct the embeddings, rather than having an LLM do it for you. However, human steganography detection algorithms have been deployed for decades, meaning that the field of cybersecurity is “no worse off” than before the investigation into LLM-based steganography systems. Despite this meaning our implementation, and therefore the study at large, was a failure, this is also a win for national security, as well as the paranoid (ourselves included) who believe in the eventuality that AI will overtake humanity.

Hypothetical speaking, if this type of steganographic messaging was capable of consistently occurring, several extensions to the implementation of these algorithms would arise. The first one that comes to mind is parallelism. If we can have one LLM generated message for a back and forth between Alice and Bob, why stop there (if feasible). One could have numerous instances of embedderLLMs running simultaneously creating a variety of relevant stories that could be sent between multiple bot accounts which may be conversing within a group chat on social media or in a discord server. On top of this, more realism could be achieved by properly scheduling the messages to occur at “humane time”. This means that conversations shouldn’t be seemingly happening 24/7, maybe messaging could stop around 10 or 11 pm and continue again at 7 or 8 am the next day. Not only does this make the messaging more inconspicuous, but it could possibly allow the AI to continuously generate conversations and have them on standby or possibly take a longer time to generate an even more realistic or feasible story.

6 RESOURCES

[1] D. Gligoroski, M. Raikwar, and S. K. Jha, "An LLM Framework For Cryptography Over Chat Channels," *arXiv.org*, 2025. <https://arxiv.org/abs/2504.08871> (accessed May 17, 2026).

7 APPENDIX

Appendix A:

Table 3: A table of top- f parameters for achieving probabilities less than 2^{-sec} for a used token to be produced with parameters outside the optimal ranges.

		E	A	S	F
	p_l	0.1041442	0.0651738	0.0515760	0.0197881
Table					
	l	1	2	3	4
sec	16	17	25	29	60
	32	34	49	58	120
	48	51	73	87	179
	64	68	97	116	239
	96	102	146	174	358
	128	136	194	232	477

Appendix A Table 1: Lookup guide found in paper

```
# Top Table
TOP_F_TABLE = {
    (16, 1): 16, (16, 2): 17, (16, 3): 25, (16, 4): 29,
    (32, 1): 32, (32, 2): 34, (32, 3): 49, (32, 4): 58,
    (48, 1): 51, (48, 2): 73, (48, 3): 87, (48, 4): 179,
    (64, 1): 68, (64, 2): 97, (64, 3): 116, (64, 4): 239,
    (96, 1): 102, (96, 2): 146, (96, 3): 174, (96, 4): 358,
    (128, 1): 136, (128, 2): 194, (128, 3): 232, (128, 4): 477
}
```

Appendix A Table 2: Lookup guide implementation in python

The numbers inside the grid (17, 60, 477, etc.) are the actual top- f values. This is the maximum number of times the algorithm is allowed to incrementally bump up the Temperature (T) to try and find a matching word before it gives up and triggers the brute-force reset.

As you move across the columns, the rarest letter becomes much harder to find (from 'E' at 10% probability down to 'F' at 1.9%). Because it's harder to find, the algorithm needs a much higher top- f limit (more retries) to successfully embed it.

As you move down the rows, you are demanding higher cryptographic security (lower chance of breaking indistinguishability). To guarantee this, the algorithm is granted a much larger top- f allowance, ensuring it exhausts every possible minor temperature adjustment before it resorts to expanding the token pool (k), which is what actually breaks the human-like text boundaries.

Appendix B: Helper function to retrieve Top k Tokens

```
# Function to retrieve Top k Tokens
# -- Parameters --
# LLM: model type desired
# Topic: topic of story
# Story: Previous story to build off of
# T: Desired temp
# k: top k token count
def top_k_token_retriever(LLM, Topic, Story, T, k):

    # Attempt to generate token with except catch
    try:
        # Call Local Model and retrieve top k candidates
        prompt = f"Topic: {Topic} \n\nContinue this story:\n{Story}"

        # Initiate chat completion API call
        reponse = client.chat.completions.create(

            model=LLM,
            messages=[{"role": "user", "content": prompt}],
            max_tokens=1,
            temperature=T,
            logprobs = True,
            top_logprobs = k
        )

        # Extract & log tokens (navigate JSON struct)
        Y_topk = []

        for prob in response.choices[0].logprobs.content[0].top_logprobs:
            Y_topk.append(prob.token)

        return Y_topk

    # Error Handling
    except Exception as e:
        print(f"Error communicating with Ollama: {e}")
        return []
```

This is where the magic happens. This helper function is the actual physical implementation of the API call which prompts the LLM to generate our top k probable tokens. The function builds a prompt that enforces the contextual rules set out by the algorithm. It feeds the LLM both the general Topic (e.g., “Syzygies of Planets”) and the Story generated so far, so the LLM knows exactly what context it is trying to continue. It then extracts the top_logprobs token list from the API call and appends them to Y_topk and returns that.

Appendix C: Helper function

```
# Function to check story if Char token fits the placement
# -- Parameters --
# Y_topk: list of generated top k tokens
# Story: current generated story
# Ci: current desired character
# bi: current desired index
def token_pos_check(Y_topk, Story, Ci, bi):

    # create placeholder list
    Y_good = []

    # Loop through and check each Ci
    for token in Y_topk:
        Story_test = Story + token

        # Check to see if added token reaches desired length
        if len(Story_test) > bi:

            # Check to see if token matches desired
            if Story_test[bi].upper() == Ci.upper():
                Y_good.append(token)

    return Y_good
```

This is a helper function I (Raiyaan) coded to accrue all top probable tokens generated by the API call accomplished by Appendix B that are compatible for insertion into the story. It acts as the Cryptographic Filter. The function essentially creates a local copy of the current story within the function and checks to see if any of the top probable tokens can successfully embed the next C char in the correct b position. It then returns this list.

Appendix D: Results of 5-character LLM Embedding

=== CSMC 443 Final Project ===

By: Raiyaan Tareen, Aidan Denham

Password : test

Plaintext: hi

Topic : Tell me about the english alphabet.

Generating steganographic story ...

[TIMING] Job started at 2026-05-18T20:02:51

[1/52] char='N' pos=19 story_len=25 elapsed=137.86s

Absolutely"OkayDefinitely

[2/52] char='O' pos=35 story_len=42 elapsed=16.86s

Absolutely"OkayDefinitelyAlrightabsolutely

[3/52] char='F' pos=54 story_len=62 elapsed=83.30s

Absolutely"OkayDefinitelyAlrightabsolutelyHelloGreatDefinitely

[4/52] char='N' pos=72 story_len=78 elapsed=46.44s

Absolutely"OkayDefinitelyAlrightabsolutelyHelloGreatDefinitelyWowYouDefinitely

[5/52] char='T' pos=90 story_len=91 elapsed=7.60s

Absolutely"OkayDefinitelyAlrightabsolutelyHelloGreatDefinitelyWowYouDefinitelyWow"HelloJust

Appendix E: Test of Encryption and Decryption using Mock Embedder

=== Test: Character Placement and Decryption (Arbitrary Plaintext Sizes) ===

```
from embedder import LLMAuthenticatedEncryption
from tests import verify_story_correctness

class RecordingPositionEmbedder:
    def __init__(self):
        self.characters = None
        self.positions = None
    def embed(self, topic, initial_story, characters, positions, temperature, top_k, security_level):
        self.characters = list(characters)
        self.positions = list(positions)
        story = ['x'] * (positions[-1] + 1)
        for char, pos in zip(characters, positions):
            story[pos] = char
        return ''.join(story)

test_cases = [
    ('tiny', b'A'),
    ('small', b'Hello!'),
    ('medium', b'Attack at dawn, general.'),
    ('large', b'The quick brown fox jumps over the lazy dog.' * 4),
    ('bytes', bytes(range(32))),
]

for label, plaintext in test_cases:
    password = f'test-password-{label}'
    embedder = RecordingPositionEmbedder()
    story, ciphertext = LLMAuthenticatedEncryption(
        2, 1, int(), story_correctness(story, password, plaintext), plaintext, topic=
    )
    --- tiny ---
    Plaintext : 'A'
    Ciphertext : 372C20CBCB1EEB7D846CCED884414C1E6B550951CCE7A1BD5C (50 hex chars
    embedded across 872-char story)
    Decrypted : 'A'

    --- small ---
    Plaintext : 'Hello!'
    Ciphertext : 8EA045B4FBFA83B2391AD82222B774CA5602B45786320B597390E6100F4D (60 hex
    chars embedded across 1045-char story)
    Decrypted : 'Hello!'

    --- medium ---
    Plaintext : 'Attack at dawn, general.'
    Ciphertext :
    9E0FBD063F14D5B02AA2A877572AA57375A8267F92C1414634D4841FDB18101D2AFDD3A8EBE4768B4A8
    C4726FC19CCC7 (96 hex chars embedded across 1685-char story)
    Decrypted : 'Attack at dawn, general.'
```

--- large ---

Plaintext : 'The quick brown fox jumps over the lazy dog. The quick brown fox jumps over the lazy dog.
The quick brown fox jumps over the lazy dog. The quick brown fox jumps over the lazy dog. '

Ciphertext :

785ACA8F773AA2053A8A7FCCC705642A40C4D66D7DF88A6D2A9902A0C45D41FDD61D7D186E0BBADF
B417E20290F65D6D5A4618E2D76CD6804E5340164A88969A0B1BD548D08BFA456D6243464772BF244BC59
77C674BB4A5197D32748CD06ABB7230E48A343D9AD51CCCD73BE5C2E11C646EB4388F2C440BBC18B59
04E40AE109D04AB648B3E33289279F1D168095EAA84CE349667860AB60D23F086149700CD425597EE8AE4
26D35274CEA710742E9AFE70D91C96DA19851C706E6277C657DF7C6F9B05393243C16CDDF05D (408 hex
chars embedded across 7151-char story)

Decrypted : 'The quick brown fox jumps over the lazy dog. The quick brown fox jumps over the lazy dog.
The quick brown fox jumps over the lazy dog. The quick brown fox jumps over the lazy dog. '

--- bytes ---

Plaintext :

"\x00\x01\x02\x03\x04\x05\x06\x07\x08\t\n\x0b\x0c\r\x0e\x0f\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\
\x1c\x1d\x1e\x1f"

Ciphertext :

EB35D060F9B9AA1C23CA9BC90A92AC6BB0D5CDA0D7841128DC6ECCE4ADC812B3534FEA599E9B3CB4
268696F653A3694062DF55B54FE46A66 (112 hex chars embedded across 1960-char story)

Decrypted :

"\x00\x01\x02\x03\x04\x05\x06\x07\x08\t\n\x0b\x0c\r\x0e\x0f\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\
\x1c\x1d\x1e\x1f"